UNIT 4 · TOPIC 4.3

Endpoint Hardening Controls

Assignment, Answer Key

Teacher answer key, 32 points total.

Practice Questions

4.3.a · Device hardening

1. A network administrator sets up a new router and immediately connects it to the internet without changing any settings. What is the PRIMARY security risk? (1 pt)

- ○ A. The router will advertise its network name to neighbors
- ○ B. The router's firmware is probably out of date, which is the only real risk
- ○ C. Attackers will be able to see the router's MAC address
- **● D. Default credentials allow any attacker to log in as administrator**

Why: Factory defaults like admin/admin are published in manuals and built into attack tools and botnets (Mirai). Anyone who can reach the login page owns the router. Outdated firmware is also a risk, which is why hardening includes updating it, but the default password is the immediate, trivially exploited one.

2. Which action BEST describes the principle of least privilege when hardening a workstation? (1 pt)

- ○ A. Giving users admin rights only on their own workstation
- ○ B. Removing all user accounts and using one shared admin account that IT controls
- ○ C. Letting users request admin rights, granted automatically for 24 hours
- **● D. Granting each user account only the permissions needed for their job functions**

Why: Least privilege means each account gets only what its work requires. Local admin rights let malware a user runs install itself system-wide; a shared admin account destroys accountability; and automatic temporary admin is still unreviewed admin.

3. Select ALL items that are part of device hardening. (Choose all that apply.) (3 pts)

- **■ A. Change default passwords**
- **■ B. Disable services not needed for the device's function**
- **■ C. Enable full-disk encryption**
- □ D. Enable Telnet for easy remote access

Why: Changing defaults, disabling unneeded services, and enabling encryption are all hardening steps. Telnet is an unencrypted protocol that increases attack surface.

4. What is 'configuration drift' in the context of device hardening? (1 pt)

- ○ A. When a device physically moves to a new network location
- **● B. The gradual accumulation of unauthorized or unintended configuration changes over time**
- ○ C. The planned update of firmware to a new version
- ○ D. The process of migrating from one security baseline to another

Why: Configuration drift occurs when small changes accumulate and unknowingly re-open attack surface that was previously closed.

5. Name the term for the total set of ways an attacker could interact with or exploit a device. (1 pt)

Acceptable answers: attack surface; attack surface area; surface area

Why: The attack surface is what hardening aims to reduce by removing unnecessary entry points.

4.3.b · Antivirus and endpoint protection

6. An administrator wants to detect a brand-new, customized piece of malware that does not match any known virus definition database. Which detection method is MOST likely to identify this threat? (1 pt)

- ○ A. Signature-based detection
- **● B. Heuristic or behavioral analysis**
- ○ C. Hashing-based verification
- ○ D. Passive packet sniffing

Why: Behavioral and heuristic analysis look at the program's actions or structure to identify anomalies rather than matching static byte sequences.

7. What is a primary drawback of relying heavily on behavioral analysis for malware detection? (1 pt)

- ○ A. It cannot detect zero-day exploits.
- ○ B. It requires a constantly updated signature database to function.
- ○ C. It only operates when the computer is disconnected from the network.
- **● D. It may generate a higher rate of false positives on legitimate administrative software.**

Why: Behavioral monitoring can flag benign administrative tools that perform system-level operations (like registry edits or process injections) as suspicious, raising false alarms.

8. Select ALL functions that a host-based firewall typically performs. (Choose all that apply.) (3 pts)

- **■ A. Blocking unauthorized inbound connections to local services**
- **■ B. Preventing unauthorized outbound connections from local malware to command servers**
- □ C. Scanning local files for known virus signatures
- **■ D. Filtering traffic specifically for the device on which it is installed**

Why: Host firewalls block unauthorized incoming and outgoing connections for a single endpoint. Scanning files for signatures is the job of antivirus software.

9. What is the term for a security alert that incorrectly flags a harmless file or action as malicious? (1 pt)

Acceptable answers: false positive; false-positive

Why: A false positive occurs when normal activity is misidentified as a threat.

10. An attacker abuses PowerShell, a legitimate pre-installed Windows tool, to download and run malicious code instead of using custom malware. What is this technique called? (1 pt)

- **● A. Living-off-the-land**
- ○ B. Signature spoofing
- ○ C. Polymorphic encoding
- ○ D. Heuristic bypass

Why: Living-off-the-land attacks abuse legitimate, pre-installed system tools, making them harder for both signature and behavioral engines to distinguish from normal administrative activity.

11. How does Endpoint Detection and Response (EDR) primarily differ from traditional antivirus? (1 pt)

- ○ A. EDR only works on servers, never on laptops or desktops
- **● B. EDR continuously records endpoint activity and gives responders tools to investigate and contain threats retroactively, beyond just blocking at scan time**
- ○ C. EDR replaces the need for any signature-based scanning

○ D. EDR requires no central console and runs independently on each device

Why: EDR builds a searchable history of endpoint activity and adds response tools like remote isolation, going beyond antivirus's block-at-scan-time model.

4.3.c · Patching and regular updates

12. What is a 'zero-day' vulnerability? (1 pt)

○ A. A vulnerability that was patched on the same day it was reported

○ B. A vulnerability that has a patch, but zero organizations have applied it yet

● C. A vulnerability being actively exploited before any patch exists

○ D. A vulnerability with no known way to exploit it

Why: "Zero-day" means the vendor has had zero days to fix it: attackers know about it (and often exploit it) before any patch exists. Once a patch is released and systems remain unpatched, it becomes an n-day vulnerability.

13. Most real-world breaches exploit which type of vulnerability? (1 pt)

○ A. Zero-days, because no patch can stop them

● B. N-days, because a patch exists but the target has not yet applied it

○ C. Vulnerabilities with a CVSS score below 1.0

○ D. Only vulnerabilities in end-of-life software

Why: The overwhelming majority of breaches exploit n-day vulnerabilities: a patch already exists, but the target system has not applied it yet.

14. Select ALL legitimate reasons an organization might delay applying a patch immediately. (Choose all that apply.) (3 pts)

■ A. The patch has not yet been tested and could break a business-critical application

■ B. Applying the patch requires a scheduled reboot that must avoid business hours

□ C. The vulnerability does not actually exist and was invented by the vendor

■ D. The organization has thousands of endpoints requiring a staged rollout

Why: Compatibility risk, downtime, and rollout scale at enterprise size are all real reasons for a tested, staged patch process rather than instant deployment.

15. A public-facing web server has a critical CVSS 9.1 vulnerability that CISA confirms is being actively exploited in the wild right now. What should happen? (1 pt)

● A. Apply emergency out-of-cycle patching as urgently as possible

○ B. Wait for the next monthly maintenance window, since change control requires it

○ C. Take a full backup and then patch at the next scheduled window

○ D. Add a firewall rule for now and patch at the next quarterly review

Why: Critical severity, internet-facing, and actively exploited is the combination that justifies breaking the normal schedule: attackers are using it today. Change control should have an emergency path for exactly this. A temporary firewall rule or other mitigation can help while patching, but it should not replace patching for months.

16. Name the term for software that no longer receives vendor security patches, requiring compensating controls instead of patching. (1 pt)

Acceptable answers: end-of-life; end of life; eol; eol software

Why: End-of-life (EOL) software receives no further vendor patches; organizations must rely on compensating controls like network isolation.

17. Why does publishing a patch increase urgency to apply it quickly? (1 pt)

- ○ A. Publishing a patch alerts attackers that the vendor is watching them
- ○ B. Patches turn off old security features, so the system is weaker until patched
- **● C. The patch itself (or its public disclosure) reveals details attackers can reverse-engineer into a working exploit**
- ○ D. Vendors stop supporting systems that do not patch within a week

Why: Attackers compare the patched and unpatched code ("patch diffing") to find exactly what was fixed, then build an exploit for everyone who has not updated. That is why the days right after a patch release are high-risk and why fast patching matters.

Coding Assignments (Python 3)

4.3.a · Device hardening

3 pts · auto-graded on the site

Coding problem 4.3.a: Device hardening configuration auditor

Skill: Mitigate Risk · **Maps to:** 4.3.A · **Language:** Python 3 (pytest)

Problem

In a secure enterprise environment, devices must be configured according to strict security baselines before deployment. Over time, administrators or software installers may change settings, resulting in "configuration drift" that increases the device's attack surface.

Your task is to write a python function `solve(data)` that takes a target device's actual configuration and audit it against a security baseline. The function must return a list of all check names that the configuration fails, sorted alphabetically.

Function contract

Implement `solve(data)` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

A dictionary with keys `"baseline"` and `"configuration"`. Example:

```
{
  "baseline": {
    "min_password_length": 12,
    "disallowed_services": ["ftp", "telnet"],
    "firewall_enabled": true,
    "encryption_enabled": true,
    "allowed_accounts": ["admin", "operator"]
  },
  "configuration": {
    "password_length": 8,
    "services": ["ssh", "ftp"],
    "firewall_enabled": false,
    "encryption_enabled": true,
    "accounts": ["admin", "guest", "operator"]
  }
}
```

Output

A list of strings representing the names of the failing checks from the baseline, sorted alphabetically. Example output for the input above:

```
["allowed_accounts", "disallowed_services", "firewall_enabled", "min_password_length"]
```

(Explanation: password length 8 is less than 12; ftp service is disallowed; firewall is false; guest account is not allowed)

Constraints

- If an expected configuration field is missing, default values apply: - `password_length`: `0` - `services`: `[]` - `firewall_enabled`: `False` - `encryption_enabled`: `False` - `accounts`: `[]`
- Standard library only.
- Case-sensitive string matching for services and accounts.

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
def solve(data: dict) -> list[str]:
    baseline = data.get("baseline", {})
    config = data.get("configuration", {})

    failed = []

    min_pw = baseline.get("min_password_length", 0)
    actual_pw = config.get("password_length", 0)
    if actual_pw < min_pw:
        failed.append("min_password_length")

    disallowed_srv = set(baseline.get("disallowed_services", []))
    actual_srv = config.get("services", [])
    if any(s in disallowed_srv for s in actual_srv):
        failed.append("disallowed_services")

    fw_required = baseline.get("firewall_enabled", False)
    fw_actual = config.get("firewall_enabled", False)
    if fw_required and not fw_actual:
        failed.append("firewall_enabled")

    enc_required = baseline.get("encryption_enabled", False)
    enc_actual = config.get("encryption_enabled", False)
    if enc_required and not enc_actual:
        failed.append("encryption_enabled")

    allowed_acc = set(baseline.get("allowed_accounts", []))
    actual_acc = config.get("accounts", [])
    if any(a not in allowed_acc for a in actual_acc):
        failed.append("allowed_accounts")

    return sorted(failed)
```

4.3.b · Antivirus and endpoint protection

3 pts · auto-graded on the site

Coding problem 4.3.b: Endpoint Detection Engine

Skill: Mitigate Risk · **Maps to:** 4.3.A, 4.3.B, 4.3.D · **Language:** Python 3 (pytest)

Problem

Endpoint Protection Platforms (EPP) use a mixture of signature detection (known hashes) and behavioral heuristics (actions taken by the process, such as excessive file updates or network socket connections) to identify and suspend threats.

You are tasked with writing a process-auditing rule engine `solve(data)`. The engine evaluates a list of running processes against: 1. A database of known malicious file hashes (signatures). 2. Max thresholds on process behavior: `max_file_modifications` and `max_network_connections`.

The function must return a list of PIDs of all processes that violate any of these rules.

Function contract

Implement `solve(data)` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

A dictionary containing security signatures, behavioral thresholds, and a list of active processes:

```
{
  "signatures": ["d41d8cd98f00b204e9800998ecf8427e", "098f6bcd4621d373cade4e832627b4f6"],
  "behavioral_rules": {
    "max_file_modifications": 10,
    "max_network_connections": 5
  },
  "processes": [
    {
      "pid": 101,
      "name": "backup_agent.exe",
      "hash": "81dc9bdb52d04dc20036dbd8313ed055",
      "file_modifications": 8,
      "network_connections": 2
    },
    {
      "pid": 102,
      "name": "ransomware.exe",
      "hash": "d41d8cd98f00b204e9800998ecf8427e",
      "file_modifications": 2,
      "network_connections": 0
    },
    {
      "pid": 103,
      "name": "spam_sender.exe",
      "hash": "1f10c9aa0090ff91e90ff8a8bc8a9bcf",
      "file_modifications": 1,
      "network_connections": 12
    }
  ]
}
```

Output

A list of integers representing the PIDs of flagged processes, sorted in ascending order. For the example input above:

`[102, 103]`

(Explanation: PID 102 matches a known signature; PID 103 exceeds the network connections threshold)

Constraints

- Signature matching is case-insensitive (e.g., `"A1B2"` matches `"a1b2"`).
- Behavioral violations occur when a metric is *strictly greater* than the threshold (i.e. `file_modifications > max_file_modifications`).
- Standard library only.

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
def solve(data: dict) -> list:
    signatures = {s.lower() for s in data.get("signatures", [])}
    rules = data.get("behavioral_rules", {})
    max_files = rules.get("max_file_modifications", 0)
    max_net = rules.get("max_network_connections", 0)

    flagged = set()
    for proc in data.get("processes", []):
        pid = proc["pid"]
        if proc.get("hash", "").lower() in signatures:
            flagged.add(pid)
            continue
        if proc.get("file_modifications", 0) > max_files:
            flagged.add(pid)
            continue
        if proc.get("network_connections", 0) > max_net:
            flagged.add(pid)
            continue

    return sorted(flagged)
```

4.3.c · Patching and regular updates

3 pts · auto-graded on the site

Coding problem 4.3.c: Patch-prioritization engine

Skill: Mitigate Risk · **Maps to:** 4.3.B, 4.3.C · **Language:** Python 3 (pytest)

Problem

A patch-management team needs to triage a queue of pending vulnerabilities and decide how urgently each must be addressed. Each vulnerability has a CVSS severity score, a flag for whether it is being actively exploited in the wild, a flag for whether the affected asset is internet-facing, and a flag for whether the affected software is end-of-life (no patch will ever be released). Build the triage engine that assigns one urgency label to each vulnerability.

Function contract

Implement `def solve(vulns: list) -> list` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

A list of dicts, each with keys: `id` (str), `cvss` (float, 0.0-10.0), `exploited_in_wild` (bool), `internet_facing` (bool), `end_of_life` (bool).

Output

A list of dicts, one per input vulnerability (same order as input), each with keys `id` (str, copied from input) and `urgency` (str), where `urgency` is one of: `"emergency"`, `"scheduled"`, `"routine"`, `"compensate"`.

Decision rule (apply in this exact order, first match wins, per vulnerability)

1. `end_of_life` is `True` → `"compensate"` (no patch will ever exist; patching is not an option regardless of severity or exploitation status). 2. Else, `exploited_in_wild` is `True` AND `internet_facing` is `True` → `"emergency"`. 3. Else, `cvss >= 9.0` AND `internet_facing` is `True` → `"emergency"` (a working exploit is likely imminent for a critical, internet-facing

flaw even without confirmed in-the-wild activity). 4. Else, `cvss >= 9.0` → `"scheduled"` (critical severity but not internet-facing). 5. Else, `cvss >= 7.0` AND `internet_facing` is `True` → `"scheduled"`. 6. Else → `"routine"`.

Rule order matters: `end_of_life` is checked first and overrides every other condition, an end-of-life system is never `"emergency"` or `"scheduled"` even at CVSS 10.0, because patching is not a valid remediation at all.

Examples

```
[{"id": "V1", "cvss": 9.1, "exploited_in_wild": True, "internet_facing": True, "end_of_life": False}]
    -> [{"id": "V1", "urgency": "emergency"}]

[{"id": "V2", "cvss": 8.8, "exploited_in_wild": False, "internet_facing": False, "end_of_life": True}]
    -> [{"id": "V2", "urgency": "compensate"}]

[{"id": "V3", "cvss": 4.2, "exploited_in_wild": False, "internet_facing": False, "end_of_life": False}]
    -> [{"id": "V3", "urgency": "routine"}]
```

Constraints

- Deterministic, offline, standard library only.
- Preserve input order in the output list.
- Exactly one urgency label per vulnerability.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
def solve(vulns: list) -> list:
    results = []
    for v in vulns:
        vid = v["id"]
        cvss = v.get("cvss", 0.0)
        exploited = v.get("exploited_in_wild", False)
        internet_facing = v.get("internet_facing", False)
        end_of_life = v.get("end_of_life", False)

        if end_of_life:
            urgency = "compensate"
        elif exploited and internet_facing:
            urgency = "emergency"
        elif cvss >= 9.0 and internet_facing:
            urgency = "emergency"
        elif cvss >= 9.0:
            urgency = "scheduled"
        elif cvss >= 7.0 and internet_facing:
            urgency = "scheduled"
        else:
            urgency = "routine"

        results.append({"id": vid, "urgency": urgency})

    return results
```